

What Makes Vect Different

A compiled language where scientific computing is the syntax — not a library

⚡ d/dx is an operator · Vectors are literals · Compiles to native .exe

d/dx

NATIVE OPERATOR

159

TESTS PASSING

LLVM

NATIVE CODEGEN

v3

CURRENT VERSION

THE CORE IDEA

Every other language makes you import science.

Vect builds it in.

Open Python, NumPy, MATLAB, Julia — all of them treat scientific computing as an add-on. You import a library. You learn its API. The seams show everywhere: `np.dot()`, `sp.diff()`, `np.linalg.solve()`. You're always one layer away from the math.

✗ EVERY OTHER LANGUAGE

```
import numpy as np
import sympy as sp

v = np.array([1, 2, 3])
A = np.array([[1,2],[3,4]])

x = sp.Symbol('x')
df = sp.diff(x**2 + 3*x, x)
val = float(df.subs(x, 2))
```

✓ VECT

```
var v = [1, 2, 3]
var A = [[1,2],[3,4]]

sym f(x) = x**2 + 3*x
var val = eval(d/dx(f(x)), x=2.0)
```

In Vect, `d/dx` is an operator — part of the grammar, like `+` or `*`. `[1,2,3]` is a vector literal, not a wrapped array. `@` is matrix multiply. There is nothing to import. The language IS the library.

These aren't features. They're language design decisions.



`d/dx` as a native operator

Symbolic differentiation with `d/dx(f(x))` — not a function call, not a method. An operator. Returns a symbolic expression you can print or evaluate.



`integral()` as a built-in

`integral(f(x), "x", 0, 3)` gives you the exact definite integral. No numerical approximation. Exact symbolic answer.



Vectors as literals

`[1.0, 2.0, 3.0]` is a vector the same way `42` is an integer. Addition, dot product, scaling — all native operators.



Matrix operations in syntax

`A @ B` multiplies matrices. `T(A)` transposes. `det(A)`, `inv(A)`, `solve(A,b)` — all built-in.



`plot()` in one line

`plot(f(x), x, -3.14, 3.14)` renders any function to a PNG. No matplotlib setup. No figure objects. One line.



Type inference

Functions don't need annotations. `fn add(a, b) { return a + b }` works for ints and floats. The compiler figures it out.



Compiles to a real .exe



Native Jupyter kernel

`vect build prog.vect -o prog` produces a standalone native executable via LLVM — the same backend as Clang, Rust, and Swift.

Run Vect code in notebooks. Variables persist across cells. `vect notebook` installs the kernel and launches Jupyter.

Real programs. Real output. One file.

Vectors — no imports, just syntax

```
var force      = [0.0, -9.81, 0.0]
var velocity   = [10.0, 15.0, 0.0]
var resultant  = force + velocity
print(resultant)      # [10, 5.19, 0]
print(force · velocity) # -147.15 (dot product)
print(norm(velocity))  # 18.027 (magnitude)
```

Matrices — linear algebra as first-class syntax

```
var A = [[1.0, 2.0], [3.0, 4.0]]
var B = [[5.0, 6.0], [7.0, 8.0]]

print(A @ B)      # [[19,22],[43,50]] matrix multiply
print(T(A))       # [[1,3],[2,4]]      transpose
print(det(A))     # -2.0               determinant
print(inv(A))     # [[-2,1],[1.5,-0.5]] inverse

var b = [5.0, 6.0]
print(solve(A, b)) # solution to Ax = b
```

Symbolic calculus — d/dx is just syntax

```
sym f(x) = x**2 + 3*x + 1

var df = d/dx(f(x))
print(df)      # 2*x + 3 (symbolic)
print(eval(df, x=2.0)) # 7.0 (numeric)

# Integration — also built in
```

```
var area = integral(f(x), "x", 0.0, 3.0)
print(area)                                # 19.0
```

Physics — real science in 5 lines

```
# Projectile motion
sym height(t) = 20.0*t - 4.9*t**2

var velocity = d/dt(height(t))           # 20.0 - 9.8*t
print(velocity)
print(eval(velocity, t=2.041))           # ≈ 0 (peak height)
plot(height(t), t, 0.0, 4.5, "Projectile Path") # PNG saved
```

f-strings + type inference

```
# No type annotations needed
fn describe(name, value) {
    print(f"{name} = {value} (type inferred)")
}

describe("score", 95)                    # score = 95 (type inferred)
describe("pi", 3.14159)                   # pi = 3.14159 (type inferred)
```

Multi-file imports

```
import "stdlib/physics.vect"
import "stdlib/vectors.vect"

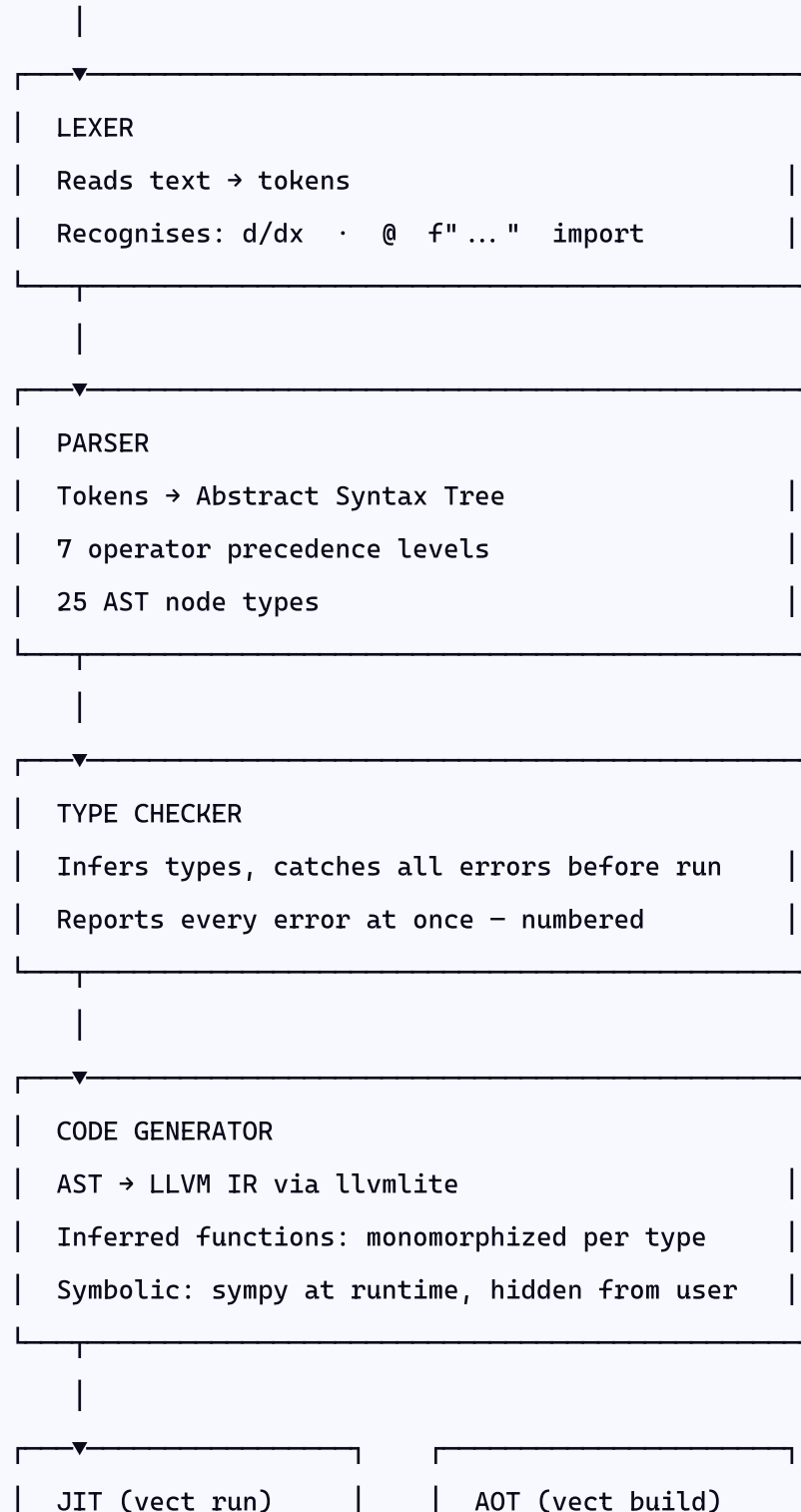
var scores = [88.0, 92.0, 95.0, 78.0]
print(f"Average: {vec_mean(scores)}")    # Average: 88.25

var ke = kinetic_energy(70.0, 10.0)
print(f"KE = {ke} Joules")               # KE = 3500.0 Joules
```

This is a real compiler. Not an interpreter.

Vect is not a scripting language with scientific helpers bolted on. It is a genuine compiled language with a complete pipeline — the same kind used to build Clang, Rust, and Swift.

source.vect



| Runs instantly |

| Native .exe output |

The **LLVM backend** is the same one used by Apple's Swift compiler, Mozilla's Rust compiler, and LLVM's own Clang. Vect uses it via **llvmlite** — a Python binding that requires zero C++ toolchain setup. **pip install llvmlite** is all you need.

Vect vs the world

FEATURE	PYTHON + NUMPY	JULIA	MATLAB	VECT
<code>d/dx</code> as syntax	✗ Library	✗ Library	✗ Toolbox	✓ Native
Integration as syntax	✗ Library	✗ Library	✗ Toolbox	✓ Native
Vector literals	✗ <code>np.array()</code>	✓	✓	✓ Native
Compiles to <code>.exe</code>	✗	✓	✗	✓
Zero-import scientific work	✗	✗	Partial	✓
Jupyter kernel	✓	✓	✓	✓
Type inference	✓	✓	✗	✓
Built from scratch	✗	✗	✗	✓ Hand-written

The key difference: In every other language, scientific operators are library functions with names like `sympy.diff()` or `numpy.linalg.solve()`. In Vect they are *syntax* — `d/dx`, `@`, `·`, `integral()`. This is not a cosmetic difference. It changes how you think when you write the code.

Up and running in 3 minutes

1. Install

```
git clone https://github.com/Eddiegah/Vect.git
cd Vect
py -3.11 -m venv venv
venv\Scripts\pip install -r requirements.txt
venv\Scripts\pip install -e .
```

2. Run the demo

```
venv\Scripts\vect run examples\demo.vect
```

3. Try the REPL

```
venv\Scripts\vect

vect> var v = [1.0, 2.0, 3.0]
vect> print(v · [4.0, 5.0, 6.0])
32.0
vect> sym f(x) = x**3 - x
vect> print(eval(d/dx(f(x)), x=2.0))
11.0
```

4. Build a native executable

```
venv\Scripts\vect build examples\fibonacci.vect -o fib
.\fib.exe
```

```
0 1 1 2 3 5 8 13 21 34 55
```

5. Open in Jupyter

```
venv\Scripts\python -m src.kernel install  
venv\Scripts\pip install jupyter  
venv\Scripts\jupyter notebook examples\vect_demo.ipynb
```

Requirements: Python 3.11 (3.9–3.12 supported). No C++ compiler needed for JIT mode. gcc (MSYS2) needed only for `vect build`. Works on Windows, macOS, and Linux.

10 source files. A complete language.

FILE	WHAT IT DOES	LINES
<code>lexer.py</code>	Tokenizer — handles d/dx, :, @, f"...", import	~250
<code>ast_nodes.py</code>	25 AST node dataclasses	~220
<code>parser.py</code>	Recursive-descent parser, 7 precedence levels	~650
<code>type_checker.py</code>	Static type inference + error collection	~650
<code>codegen.py</code>	LLVM IR code generator with monomorphization	~1200
<code>runtime.py</code>	Python ctypes callbacks for vec/mat/sym/plot	~600
<code>aot.py</code>	AOT compiler — object file + gcc linker	~250
<code>pipeline.py</code>	Import resolver + compilation pipeline	~120
<code>repl.py</code>	Interactive REPL with session persistence	~150
<code>kernel.py</code>	Jupyter kernel — all features in notebooks	~200

Plus 3 stdlib files (`physics.vect` , `vectors.vect` , `mathlib.vect`), 16 example programs, 159 tests, GitHub Actions CI, and VS Code syntax highlighting.

"Most languages treat scientific computing as a library."

Vect makes it grammar.

d/dx is not a function. It is a word in the language."

Vect v3

Built by **Eddie Gah** · github.com/Eddiegah/Vect

Python 3.11 · llvmlite · sympy · matplotlib · ipykernel · 159 tests passing